ПРУФ-ИТ / PROOFIT

Архитектура и инженерное обоснование

Высоконагруженное ядро на Rust. Проверяемые границы доменов и вычислений.

Редакция документа: 11 октября 2026. Архитектурные утверждения связаны с указанными модулями; числовые результаты — с протоколами измерений, их датами и областью применимости. Обновление документа не означает повторного проведения всех экспериментов.

Роли: специалист, работодатель, явно одобренный эксперт. Окончательное решение о найме, трактовка спорного evidence и разрешение на обработку данных не передаются автономному score.

1. Назначение и системная граница

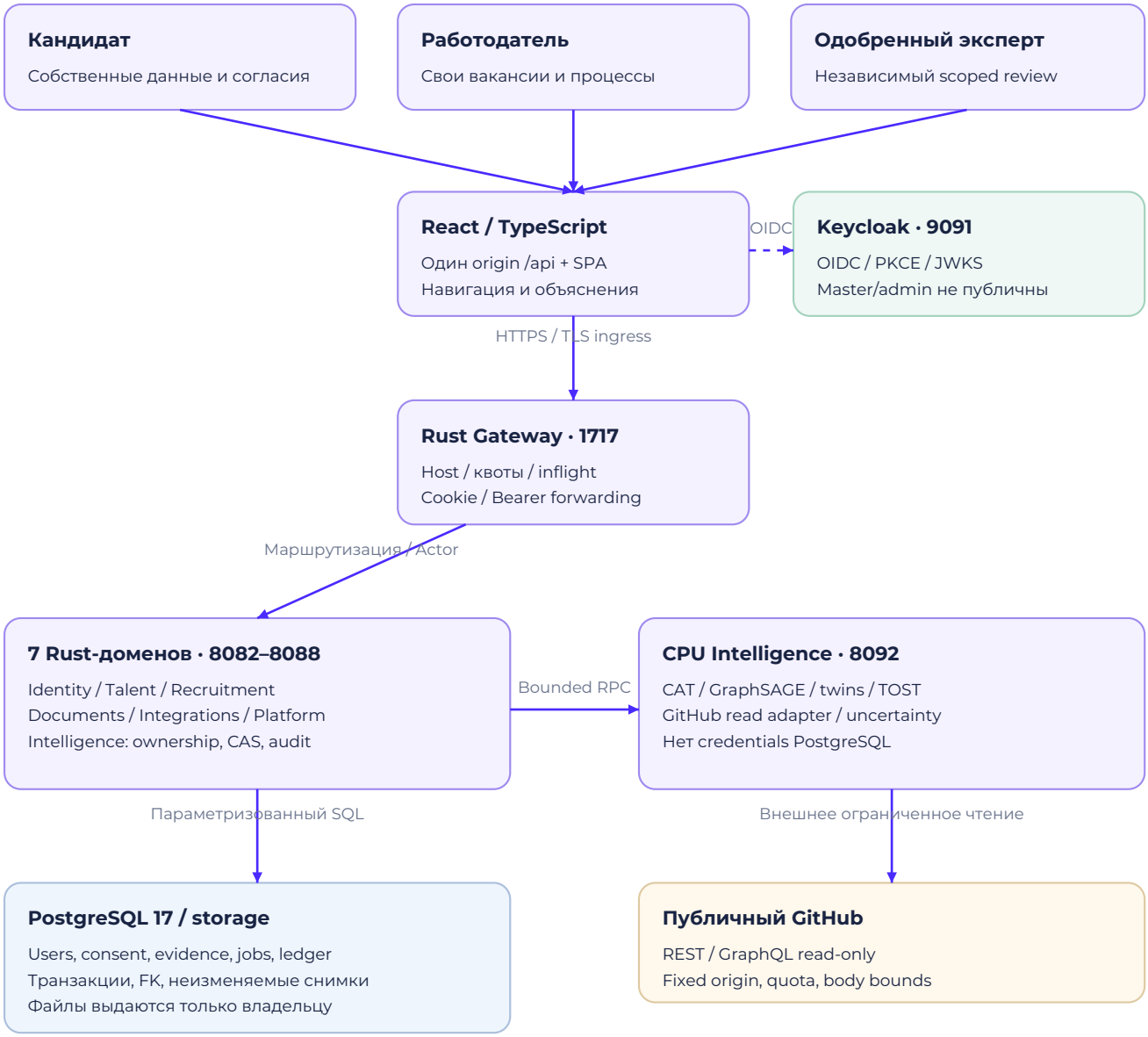
ПРУФ-ИТ — платформа обратного найма: специалист формирует профиль и проверяемые доказательства; работодатель ищет релевантность, предлагает оплачиваемые условия и получает контакты после согласия. Интеллектуальный слой объясняет совместимость, моделирует сценарии и предлагает следующий проверяемый шаг. Решение о найме остается у людей.

Три вида истины не смешиваются: пользовательское заявление, исполнимое доказательство и прогнозная гипотеза. Сертификат связан с ролью и попыткой, граф — с источником и датой, сценарий — с моделью и допущениями. Это обеспечивает проверяемость цепочки «вход → метод → решение → последующее действие».

2. Схема архитектуры: компоненты и границы доверия

Карта контейнеров отражает фактические входы, домены, владельца состояния и внешние зависимости. Стрелки обозначают конкретный вид взаимодействия, а не произвольную последовательность блоков. Основные специализированные процессы перечислены в нижней полосе; подробные порты и resource budget — в следующей таблице.

КОМПОНЕНТЫ И ГРАНИЦЫ ДОВЕРИЯ



ДОЛГИЕ / СПЕЦИАЛИЗИРОВАННЫЕ ВЫЧИСЛЕНИЯ

Processor 8081 · PDF/OCR; Workbench 8091 · Docker; Forecast 8090 · CatBoost

Контроллер очереди имеет DB-доступ; вычислительные сервисы — отдельная граница.

3. Декомпозиция и фактическое размещение

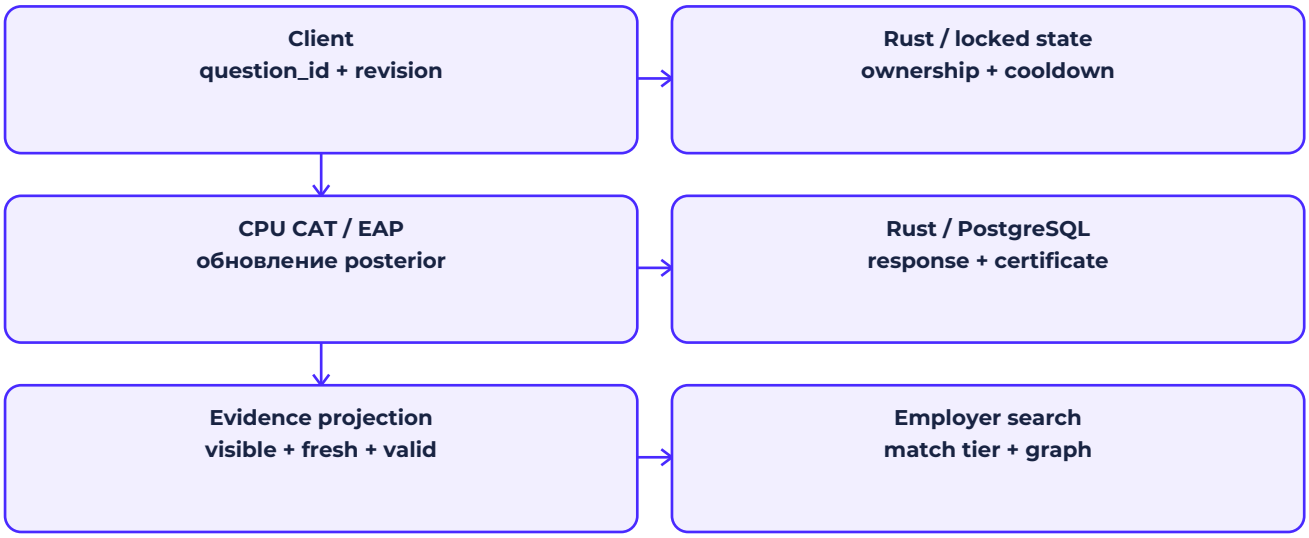
Локальный запуск — восемь процессов одного Rust-бинарника: gateway и семь доменов. Каждый домен имеет отдельный listener и пул PostgreSQL; совокупный стандартный бюджет — 4 соединения на процесс. Python intelligence не получает учетных данных PostgreSQL: доступ к данным и ownership выполняет Rust. Общая БД означает согласованную схему и междоменную связность; это доменно-декомпозированная топология, а не утверждение о независимых базах.

Домен / порт	Ответственность	Точка проверки
Gateway / 1717	Публичный вход, /api, SPA, /identity; локальные внутренние порты	src/gateway.rs; scripts/start_local.py
Identity / 8082	Учетные записи, сессии, экспорт / удаление	src/auth.rs; src/account.rs
Talent / 8083	Резюме, квалификация, проекты, консультант	src/resumes.rs; src/certification.rs
Recruitment / 8084	Вакансии, поиск, приглашения, согласия, задачи	src/matching.rs; src/interactions.rs
Documents / 8085	Загрузка, review извлечения, файлы владельца	src/documents.rs; worker/
Integrations / 8086	ATS Talantix и ограниченные внешние контракты	src/talantix.rs
Platform / 8087	Здоровье, OpenAPI и платформенные маршруты	src/api.rs
Intelligence / 8088	Durable jobs, граф, twins, provenance, OIDC	src/intelligence.rs; src/evidence.rs
CPU engine / 8092	Чистые ограниченные математические операции	intelligence/service.py
Workbench / 8091	Исполнение кода и опциональное AI-review	worker/workbench.py
Forecast / 8090	CatBoost time prior, объяснимые контрфактуалы	ml_time_forecasting/service.py

4. Транзакционный путь квалификации

Пользователь получает ровно один текущий вопрос, без правильного ответа и банка. Сервер фиксирует время показа, question_id и revision. Ответ обрабатывается под блокировкой состояния; CAS предотвращает повторное использование и параллельное продвижение. CPU-оценщик возвращает новое состояние, Rust сохраняет trace и завершение. Сертификат содержит неизменяемый снимок роли, грейда, проверенных навыков, версии метода и срока.

Повторная попытка ограничивается идентичностью кандидата, ролью и грейдом, а не только конкретным резюме. Изменение зарплаты, создание нового резюме или переключение заявленного грейда не превращает заявление в подтвержденную квалификацию. История остается в qualification_certificates и qualification_attempt_history.



5. Очереди и границы ресурсоемких операций

Загрузка PDF и OCR идут через длительные задания; результат показывается для подтверждения полей. Интеллектуальные задания имеют persisted status, lease/heartbeat и повтор после отказа. Семантическая генерация, OCR, обработка документов и Docker-run не выполняются внутри дешевого поискового запроса. Это отделяет latency-critical retrieval от expensive execution.

Gateway ограничивает inflight (по умолчанию 256), параллельные upload (2), тело запроса (20 MB), время зависимостей и открывает circuit breaker после повторяющихся сетевых отказов. Sandbox имеет 2 слота и отдельные жесткие ресурсы. Числа — конфигурационные ограничения, а не измеренная емкость системы. Дополнительные бизнес-квоты действуют на пользователя и операции.

6. Почему Rust в высоконагруженном ядре

Выбор Rust относится к частым авторизованным операциям и транзакционным состояниям: управление памятью через ownership, статическая типизация, async Tokio, Axum, параметризованный SQLx, контролируемые пулы и отсутствие пауз tracing-GC в самом Rust-ядре. Это полезные свойства для предсказуемого расхода ресурсов, но не автоматическая гарантия производительности. [RUST]

Python выбран для численных методов, SciPy, CatBoost и обработки документов: зрелая научная экосистема и исполнимые тестовые оракулы. TypeScript обеспечивает типовой браузерный контракт; PostgreSQL — атомарные бизнес-инварианты, ledger, индексируемый retrieval и согласованность. Docker изолирует недоверенный код; Keycloak реализует стандартный OIDC вместо собственной имитации identity provider.

Проверка выбора стека опирается на release-build, HTTP-контракты, bounded concurrency и latency профили. Результаты представлены в 07_Validation.pdf. Экстраполяция на миллионы сессий не выполняется: активная HTTP-конкурентность, число сессий и количество пользователей — разные величины.

7. Масштабирование без подмены доказательств

Helm-конфигурация включает доменные Deployment/HPA, отдельный intelligence engine, сетевые политики, worker и инфраструктурные интерфейсы. Проверены lint, rendered YAML и инварианты конфигурации; это не запуск Kubernetes-кластера. Перед промышленным масштабированием измеряются база данных, распределенные лимиты, очереди, внешний ingress и длительные OCR/AI-нагрузки.

Практическая схема роста: разнести генерацию и retrieval, поставить внешний TLS-ingress и ограничение на edge, пулер PostgreSQL в совместимом режиме, вынести storage, настроить единые квоты, затем увеличивать реплики по измеренным bottleneck. Последовательность сохраняет проверяемую семантику поиска и согласий. Монолитная схема БД требует контроля миграций, transaction boundaries и суммарного числа соединений.

8. Контракты, воспроизводимость и изменение

OpenAPI хранится как машинный контракт в data/contracts/openapi.json; доступен через /api/openapi.json. SQL-миграции добавляются последовательно, уже примененные не переписываются. Lock-файлы и manifest замороженных моделей фиксируют программные и модельные зависимости. Артефакты validation/reports/current связаны SHA-256 с текущим исходным кодом.

Архитектурные решения: обязательная квалификация — детерминированный CPU, AI — вспомогательное объяснение; declared skill не равен verified skill; неизвестные предпочтения не считаются отрицательными; antiplag signal не равен санкции; внешний источник не становится подтвержденным без проверенного контракта; сценарий twin не равен наблюдаемому исходу найма.

9. Политика доказательств без изменения топологии

Категоризация централизована в существующем PostgreSQL; Rust-сервисы используют одну проверяемую проекцию допуска. Расхождение declared/confirmed исключено в discovery, приглашениях и вычисляемом пуле. Контактный grant, подтверждение знаний, профессиональная аттестация и нейронная рекомендация остаются отдельными типами решения; один не подменяет другой. Миграции append-only; build.rs отслеживает изменение каталога embedded SQLx migrations и OpenAPI, поэтому выпуск содержит актуальную схему.

10. Фоновые действия и ограниченный read path

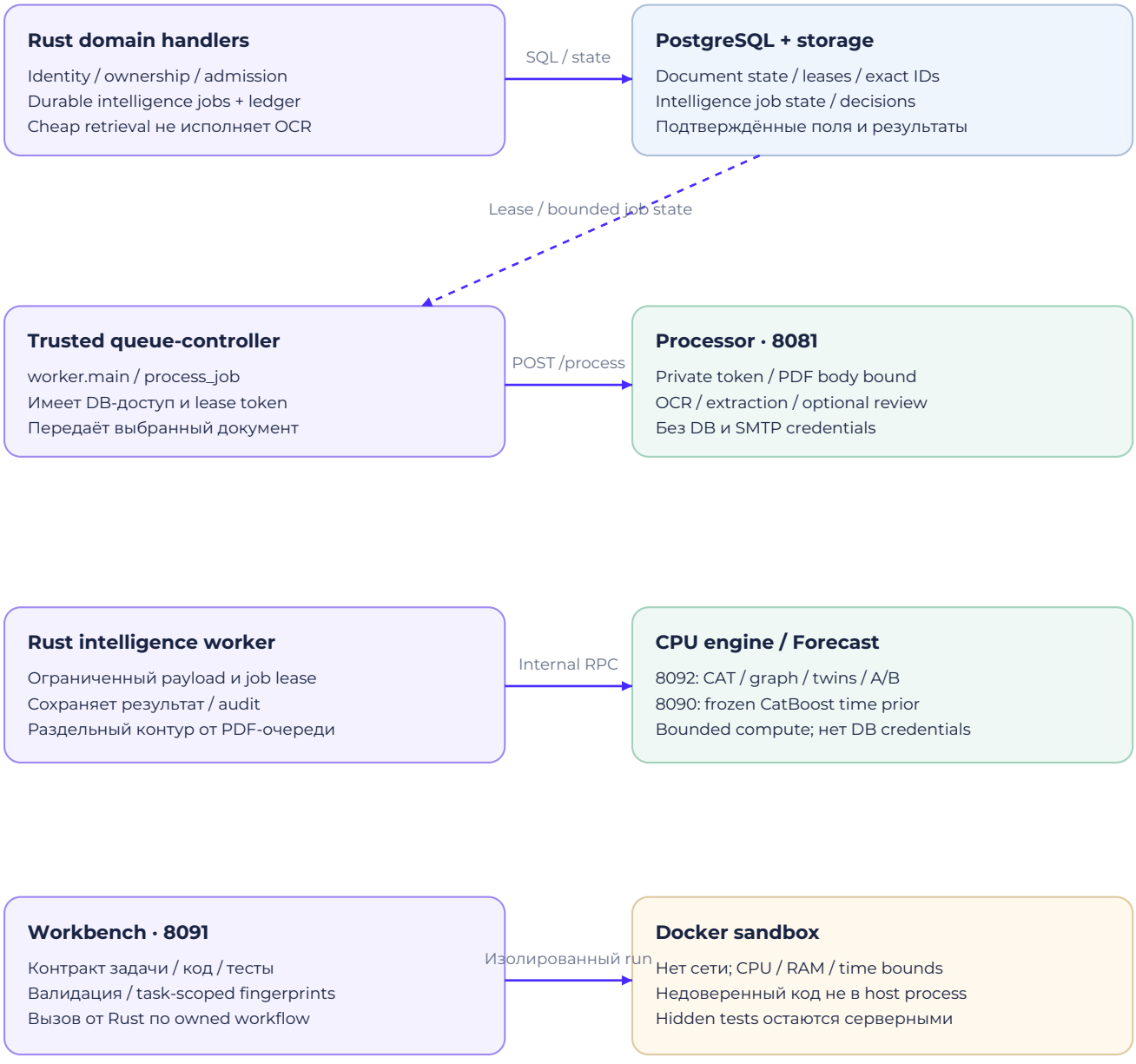
Существующий Intelligence-домен запускает persisted intelligence jobs и weekly dispatcher. Еженедельная кампания выбирается под FOR UPDATE SKIP LOCKED; company lock и проверка недавних micro_tasks не допускают повторного задания. Корреляция skill_support выполняется по конкретному req.skill: подтверждение Python не закрывает TypeScript. Кампания работает с уже согласованными взаимодействиями сторон, не создает контактное согласие.

Инвентарные категории вычисляются только в кабинете работодателя, который отображает этот график. Число доступных профилей получается из той же группировки, без второго полного прохода; личный dashboard кандидата читает только собственные счетчики. Никакого кэша допуска или задержки отзыва доказательств не введено.

11. Схема исполнения: очереди и изоляция вычислений

Доверенный контроллер PDF-очереди имеет доступ к БД и storage, но передаёт документ отдельному processor-сервису без DB/SMTP credentials. Rust intelligence worker сохраняет job state и вызывает ограниченные математические сервисы. Workbench выносит недоверенный код в Docker. Эти три контура имеют разные границы доверия; фраза «Python без БД» относится к вычислительным сервисам, не к trusted queue-controller.

ОЧЕРЕДИ, СЕКРЕТЫ И ВЫЧИСЛИТЕЛЬНАЯ ИЗОЛЯЦИЯ

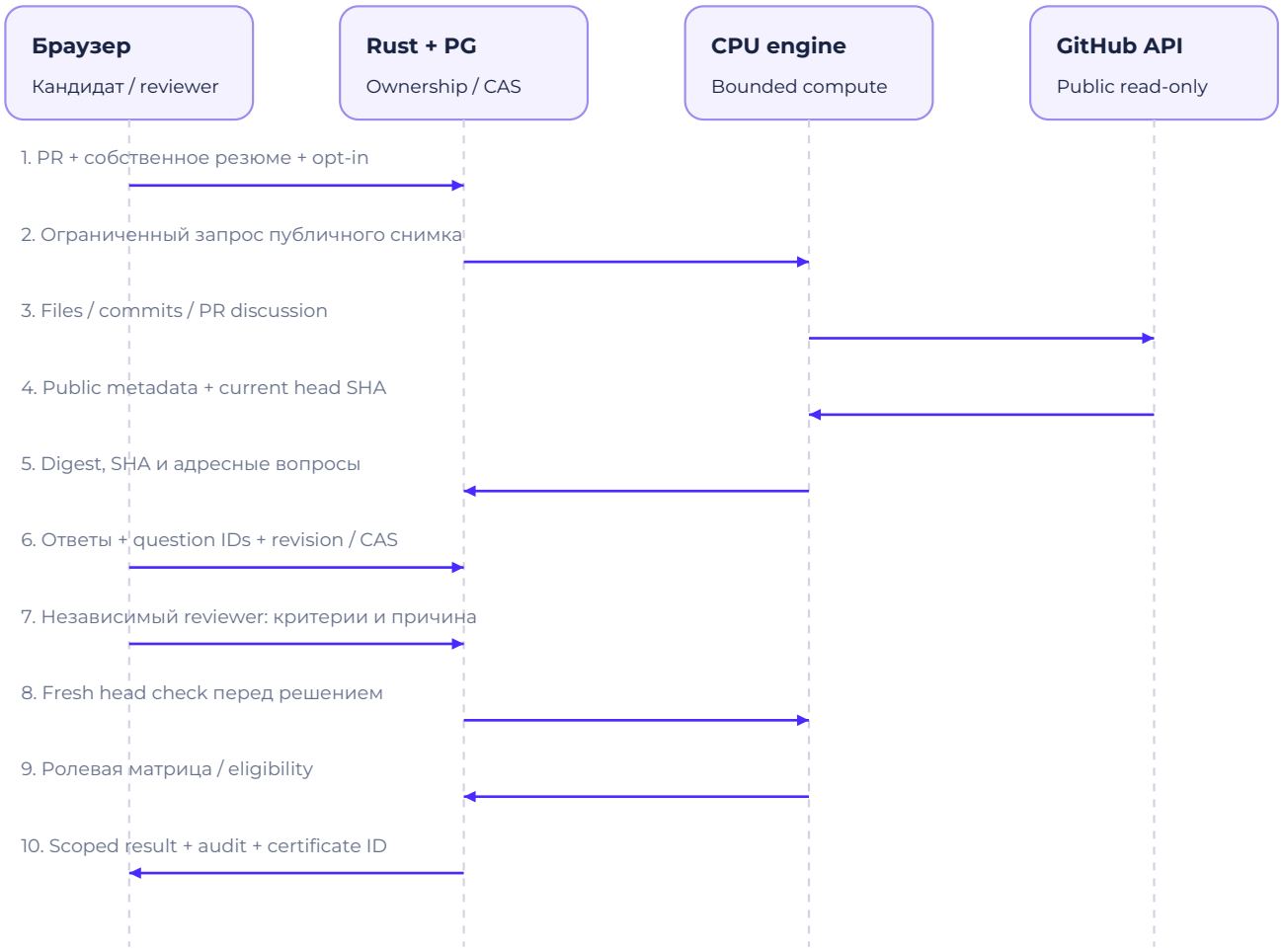


Файлы и результаты не доступны напрямую из SPA. Endpoint visibility ≠ разрешение владельца.

12. Схема взаимодействия: защита PR и ролевое решение

Кандидат и независимый эксперт используют один браузерный контур с разными identity. Подтверждение знаний, личного вклада и наблюдаемой практики проверяется раздельно. Внешнее чтение не открывает произвольный URL и не исполняет код репозитория. Перед решением сохраняется контроль head SHA.

ЗАЩИТА ПУБЛИЧНОГО PR: ВЗАИМОДЕЙСТВИЯ



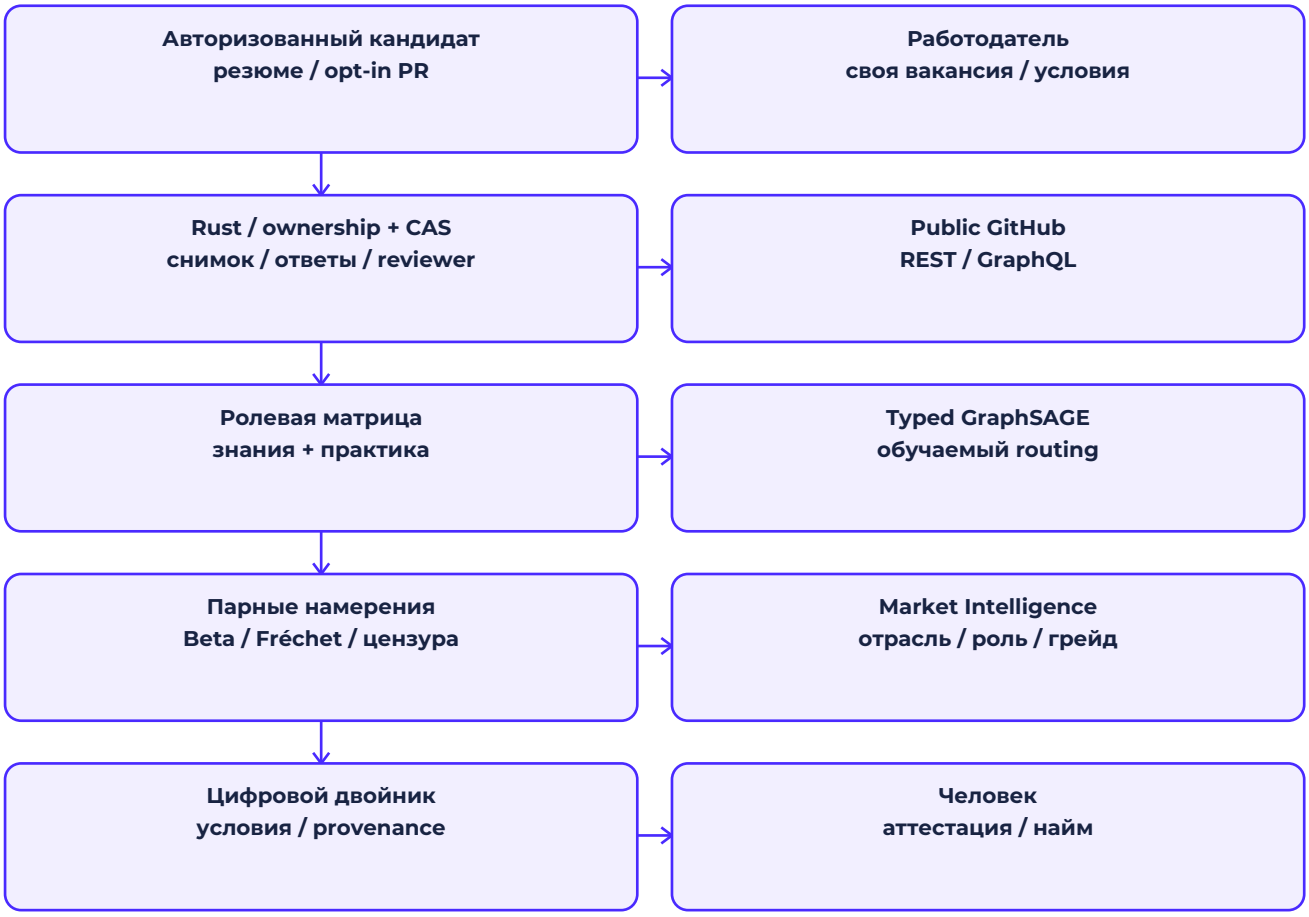
Шаг 7: другая одобренная identity; обычный работодатель не получает право аттестации.

Rust отдельно проверяет knowledge, identity, SHA, критерии и ограничения. Нет auto-grade.

13. Инженерная композиция

Платформа соединяет независимые виды информации: публичный артефакт разработки, проверку знаний роли, наблюдаемую практику, текущие условия вакансии и явно выраженные намерения обеих сторон. Каждый слой имеет собственный знаменатель, происхождение и область применимости. Математическая оценка не превращает предположение в факт квалификации.

Rust выполняет identity, ownership, транзакционные переходы и аудит. CPU Intelligence получает ограниченный набор данных без доступа к PostgreSQL. Внешний GitHub gateway не выполняет код и не принимает произвольный URL API. Ролевые и рыночные расчёты не зависят от вызова генеративной модели.



14. Проспективный ledger и проверяемая вероятность

Владелец вакансии выбирает существующего участника процесса и фиксирует прогноз до первого явно выраженного намерения каждой стороны. Rust самостоятельно вычисляет вероятность из завершённых предыдущих пар, сохраняет роль, грейд, стадию, model version и серверное время. Клиент не задаёт вероятность или исход. В реальной вакансии синтетические участники не входят в обучающий счётчик. Advisory-lock сериализует запись прогноза и намерений для одной пары/стадии; clock_timestamp фиксирует фактическое время после получения блокировки.

Записи prospective_predictions неизменяемы на уровне PostgreSQL trigger. Когда обе стороны ответили после снимка, отдельно однократно сохраняется первый парный исход в prospective_outcomes. Позднейшее изменение намерения не меняет исход оценки. Отчёт считает Brier=(p−y)², log loss, ECE по десяти фиксированным интервалам и AUC с корректными ties. Одноклассовой выборке AUC не присваивается. Joint point оценивается лишь для снимков с идентифицированной совместной оценкой; coverage показано отдельно. Vacancy/offer и role/grade blocks не смешиваются.

Синтетические записи исключаются, неполные пары считаются цензурированными. Минимум для диагностической достаточности — 120 завершённых пар и 20 наблюдений каждого класса; количество не даёт автоматического допуска к калибровке. Зависимость повторных кандидатов и вакансий сохраняется; отчёт не приписывает этим записям независимый популяционный confidence interval. Online report ограничен 500 снимками, owned export — 5000; превышение явно требует фильтра vacancy/stage или отдельного протокола, а не тихо усекает данные. Намерение не означает трудоустройство.

15. Единый авторизованный контур проверки

Проверка платформы выполняется через штатную авторизацию: по отдельно переданным данным тестовых кабинетов либо после собственной регистрации. Сессия, серверная роль и принадлежность данных проверяются обычными обработчиками. Вход в другую роль требует учетных данных ее кабинета; гостевого входа, автоматической смены identity и анонимной выдачи паролей нет. Синтетические резюме и вакансии сохраняются без пересоздания пользователей. Прежние гостевые сессии отозваны миграцией 0030 без удаления данных. Такая организация делает границы доступа частью проверяемого пользовательского маршрута. Протокол и область измерений — 11_Interface_and_Demo.pdf.